TD3 : Les Piles

Exercice 1

1) Écrire en C la fonction `factorielle` qui retourne $n!$ en utilisant une pile.

2) Écrire en C la fonction `creer_copie` qui permet de créer une copie d'une pile p.

3) Écrire en C la fonction `chercher_element` qui permet de chercher un élément e dans une pile p et affiche « l'élément existe dans la pile » si e existe et « l'élément n'existe pas dans la pile » sinon.

Exercice 2

1) Écrire la fonction `est_palindrome` qui utilise une pile pour vérifier si une chaîne de caractères est un palindrome. Par exemple, « radar » est un palindrome, mais « informatique » ne l'est pas.

2) Utiliser une pile pour inverser une chaîne de caractères. Écrire la fonction `inverser_chaine` en C qui prend une chaîne en entrée, la stocke dans une pile, puis l'affiche dans l'ordre inverse.

Exercice 3 : Parenthésage

On cherche à vérifier si une chaîne de caractères **expression** est bien parenthésée, c'est-à-dire si le nombre ainsi que l'ordre des parenthèses ouvrantes et fermantes sont corrects. On note « (» la parenthèse ouvrante et «) » la parenthèse fermante.

Écrire une fonction `parenthesage` qui admet comme argument une chaîne de caractères **expression**. Cette fonction retourne 1 si la chaîne de caractères est bien parenthésée, et 0 sinon.

- `expression = "(2+6) / (9-3)"` → `parenthesage(expression)` doit retourner 1
- `expression = "((3+6) / (5+9)"` → `parenthesage(expression)` doit retourner 0

Exercice 4

Utiliser une pile pour évaluer une expression arithmétique postfixée codée sur un tableau de caractères, en supposant pour simplifier que : tous les opérateurs sont binaires et limités à $+$, $-$, $*$ et $/$. On utilise uniquement des nombres entiers naturels.

Indication : On utilise la pile pour stocker les opérandes. Dès qu'on lit une opération, on récupère les deux opérandes en sommet de pile, on effectue l'opération et on met le résultat en sommet de pile. À la fin de la lecture de l'expression, il ne doit rester dans la pile qu'une seule valeur : le résultat du calcul.

Note : on se donne la primitive « `est_opérande` » pour tester si un caractère est opérande ou pas, de même la primitive « `effectuer_opération` » pour effectuer une opération arithmétique entre deux opérandes.